

Network Library Comparison

Document Number: D4053R0
Date: 2026-03-12
Audience: LEWG
Reply-to: Vinnie Falco vinnie.falco@gmail.com
Steve Gerbino steve@gerbino.co

Table of Contents

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. Error Handling

How it works

3. Error Propagation

How it works

4. Cancellation

How it works

5. Symmetric Transfer

How it works

6. Accept Loop

7. Echo Session

Full comparison

8. Support Machinery

What each header implements

9. Structured Concurrency

Four properties demonstrated

10. TCP Server: Full Program

11. Postcondition Analysis

Abstract

Two TCP echo servers built on different architectures - [beman.net](#)^[2] (sender/receiver, [P2762R2](#)^[1]) and [Corosio](#)^[3] (coroutine-native, built on [Capy](#)^[4]) - are compared source to source across ten dimensions. This paper is informational and proposes no action.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.
-

1. Disclosure

The authors developed [Corosio](#)^[3] and have an interest in this space. This paper proposes no action. It presents direct source-to-source comparisons so the reader can make an informed assessment.

2. Error Handling

Claim: "Sender/receiver error handling composes generically across implementations."

```
// beman.net - server.cpp
} catch (const std::exception& e) {
    std::cerr << "ERROR: " << e.what() << '\n';
}
```

```
// Corosio - echo_server.cpp
if (ec) break;
```

Every `ECONNRESET` - a routine network event - is `make_exception_ptr` + `rethrow_exception` . Two heap allocations and a table lookup per I/O error, by design.

How it works

The sender-to-coroutine bridge in `demo_task.hpp` ^[2] has no path that avoids exceptions:

```
// demo_task.hpp - sender_awaiter::set_error
template <typename Error>
auto set_error(Error&& err) noexcept -> void {
    if constexpr (std::same_as<std::decay_t<Error>, std::exception_ptr>)
        this->awaiter->error = err;
    else
        this->awaiter->error =
            std::make_exception_ptr(std::forward<Error>(err));
    this->awaiter->handle.resume();
}

// demo_task.hpp - sender_awaiter::await_resume
auto await_resume() {
    if (this->error)
        std::rethrow_exception(this->error);
    return std::move(*this->result);
}
```

`set_error` calls `make_exception_ptr` unconditionally. `await_resume` calls `rethrow_exception` unconditionally. This is not a bug; it is the only way the sender-to-coroutine bridge can surface errors through `co_await` .

The echo handler wraps its entire body in `try/catch` :

```
// beman.net - server.cpp
auto make_client(auto client) -> demo::task<void> {
    try {
        char buffer[8];
        while (auto size = co_await net::async_receive(
            client, net::buffer(buffer)))
        {
            std::string_view message(+buffer, size);
            std::cout << "received<" << size << ">" << message << ")\n";
            auto ssize = co_await net::async_send(
                client, net::const_buffer(buffer, size));
            std::cout << "sent<" << ssize << "/" << message.size()
                << ">" << std::string_view(buffer, ssize) << ")\n";
        }
        std::cout << "client done\n";
    } catch (const std::exception& e) {
        std::cerr << "ERROR: " << e.what() << '\n';
    }
}
```

In Corosio, the error code is a value. No exception. No `make_exception_ptr`. No `rethrow_exception`:

```
// Corosio - echo_server.cpp
copy::task<> do_session()
{
    for (;;)
    {
        buf_.resize(4096);
        auto [ec, n] = co_await sock_.read_some(
            copy::mutable_buffer(buf_.data(), buf_.size()));
        if (ec)
            break;

        buf_.resize(n);
        auto [wec, wn] = co_await copy::write(
            sock_, copy::const_buffer(buf_.data(), buf_.size()));
        if (wec)
            break;
    }
    sock_.close();
}
```

3. Error Propagation

Claim: "The three-channel model cleanly separates success from failure."

```
// beman.net - initiate.hpp
auto exp{co_await (
    beman::net::async_connect(client)
    | beman::net::detail::into_expected
    | beman::execution::then(
        [](auto&& e) { return std::move(e); })});
if (!exp) {
    co_yield beman::execution::with_error(exp.error());
}
```

```
// Corosio
auto [ec] = co_await sock.connect(ep);
if (ec) co_return;
```

Six lines and three adaptors to do what `if (ec)` does. The `then([](auto&& e) { return std::move(e); })` is an identity lambda - it does nothing, but the pipeline requires it to compile.

How it works

The error goes to `set_error` (failure channel). `into_expected` moves it back to `set_value` (success channel). `then` wraps it. The coroutine unwraps it. Then `co_yield with_error` puts it back in the failure channel. The error crosses the channel boundary three times to end up where it started.

The `into_expected` ^[2] implementation:

```
// demo_algorithm.hpp - into_expected
template <beman::net::detail::ex::sender Sender>
inline auto demo::into_expected_t::operator()(Sender&& s) const {
    using value_type = ex::value_types_of_t<
        Sender, ex::env<>,
        demo::detail::decayed_tuple_or_single_t,
        std::type_identity_t>;
    using error_type = ex::error_types_of_t<Sender>;
    return std::forward<Sender>(s)
        | beman::net::detail::ex::then(
            [<typename... A>(A&&... a) noexcept {
                return std::expected<value_type, error_type>(
                    std::in_place_t{}, std::forward<A>(a)...);
            })
        | beman::net::detail::ex::upon_error(
            [<typename E>(E&& e) noexcept {
                return std::expected<value_type, error_type>(
                    std::unexpected_t{}, std::forward<E>(e));
            });
}
```

4. Cancellation

Claim: “Senders provide structured cancellation that raw coroutines cannot express.”

```
// beman.net - server.cpp
while (true) {
```

```
// Corosio - tcp_server.cpp
while (!env->stop_token.stop_requested()) {
```

The sender-based server has no way to stop. `while (true)` runs forever. Timer expiry - expected control flow - is routed through `set_error`, converted to an exception, and caught by `catch (...)` { “timer fired” }.

How it works

To express “accept with a 1-second timeout” in beman.net requires four abstractions:

```
// beman.net - server.cpp
while (true) {
    try {
        auto [stream, ep] =
            co_await demo::when_any(
                net::async_accept(acceptor),
                demo::into_error(
                    net::resume_after(ctxt.get_scheduler(), 1s),
                    [](auto&&...) {
                        return ::std::error_code();
                    })
                ));
        std::cout << "ep=" << ep << "\n";
        scp.spawn(make_client(std::move(stream)));
    } catch (...) {
        std::cout << "timer fired\n";
    }
}
```

`when_any` races accept against timer. `into_error` converts the timer's success into an error. A lambda manufactures an `error_code`. `catch (...)` detects which arm won. Four abstractions for one timeout. And the loop never exits.

Corosio uses hierarchical stop tokens at three levels:

```

// Corosio - tcp_server.cpp: server level
void tcp_server::stop()
{
    if (!running_)
        return;
    running_ = false;
    impl_->stop.request_stop();
    capy::run_async(ex_, std::stop_token{})(do_stop());
}

// Corosio - tcp_server.cpp: accept level
capy::task<void>
tcp_server::do_accept(tcp_acceptor& acc)
{
    auto env = co_await capy::this_coro::environment;
    while (!env->stop_token.stop_requested())
    {
        auto& w = co_await pop();
        auto [ec] = co_await acc.accept(w.socket());
        if (ec)
        {
            co_await push(w);
            continue;
        }
        w.run(launcher{*this, w});
    }
}

// Corosio - tcp_server.cpp: worker level
capy::task<>
tcp_server::do_stop()
{
    for (auto* w = active_head_; w; w = w->next_)
        w->stop_.request_stop();
    co_return;
}

```

5. Symmetric Transfer

Claim: "Sender/receiver composition avoids unbounded stack growth."

```
// beman.net - demo_task.hpp
auto await_suspend(coroutine_handle<Promise> h) -> void {
    this->handle = h;
    ex::start(this->state);
}
```

```
// Corosio
auto await_suspend(coroutine_handle<> h, io_env const* env)
    -> coroutine_handle<>;
```

`await_suspend` returns `void`. If the sender completes synchronously inside `start()`, `set_value` calls `handle.resume()` while still on the `await_suspend` stack. Each synchronous completion adds a frame. There is no upper bound.

How it works

When `ex::start(this->state)` completes synchronously, the receiver's `set_value` runs immediately:

```
// demo_task.hpp - sender_awaiter::receiver::set_value
template <typename... Args>
auto set_value(Args&&... args) noexcept -> void {
    this->awaiter->result.emplace(std::forward<Args>(args)...);
    this->awaiter->handle.resume(); // resumes on this stack
}
```

`handle.resume()` is called while still on the `await_suspend` -> `start` -> `set_value` call chain. If the resumed coroutine `co_await`s another synchronously-completing sender, the pattern nests. There is no tail call.

This is not a demo limitation. `void` return from `await_suspend` is the only option when the sender-to-coroutine bridge does not own the resumption. Corosio's awaitables return `coroutine_handle<>`, enabling the compiler to tail-call. Stack depth is constant regardless of call chain depth.

6. Accept Loop

Claim: "Sender algorithms compose naturally to express concurrent I/O patterns."

beman.net^[2] (22 lines):

```
scope.spawn(std::invoke(
    [](auto& scp, auto& ctxt) -> demo::task<> {
        net::ip::tcp::endpoint endpoint(
            net::ip::address_v4::any(), 12345);
        net::ip::tcp::acceptor acceptor(ctxt, endpoint);

        while (true) {
            try {
                auto [stream, ep] =
                    co_await demo::when_any(
                        net::async_accept(acceptor),
                        demo::into_error(
                            net::resume_after(
                                ctxt.get_scheduler(), 1s),
                            [](auto&&...) {
                                return ::std::error_code();
                            }
                        )
                    );
                std::cout << "ep=" << ep << "\n";
                scp.spawn(make_client(std::move(stream)));
            } catch (...) {
                std::cout << "timer fired\n";
            }
        }
    },
    scope,
    context));
```

Corosio^[3] (15 lines):

```

capy::task<void>
tcp_server::do_accept(tcp_acceptor& acc)
{
    auto env = co_await capy::this_coro::environment;
    while (!env->stop_token.stop_requested())
    {
        auto& w = co_await pop();
        auto [ec] = co_await acc.accept(w.socket());
        if (ec)
        {
            co_await push(w);
            continue;
        }
        w.run(launcher{*this, w});
    }
}

```

Five abstractions to accept a connection: `std::invoke` (IILE to avoid coroutine capture), `when_any` (race accept against timer), `into_error` (convert timer success to error), `try/catch` (detect which arm won), `scope.spawn` (dispatch handler). Two abstractions in Corosio: `co_await` and `if`.

`std::invoke` is required because a coroutine lambda that captures by reference is use-after-free. The IILE passes `scope` and `context` as parameters instead. This is a coroutine safety workaround that the sender model forces by requiring the accept loop to be a lambda inside `scope.spawn`.

7. Echo Session

Claim: “The sender awaitable bridge gives coroutines seamless access to sender results.”

```

// beman.net - server.cpp
while (auto size = co_await net::async_receive(
    client, net::buffer(buffer)))

```

```

// Corosio - echo_server.cpp
auto [ec, n] = co_await sock_.read_some(
    capy::mutable_buffer(buf_.data(), buf_.size()));
if (ec) break;

```

The “seamless” bridge discards the error code. `async_receive` produces both a byte count and a status, but the coroutine only sees the byte count. The status went to `set_error` and became an exception. The programmer cannot inspect both values from the same `co_await` expression.

Full comparison

beman.net^[2] `make_client` :

```
auto make_client(auto client) -> demo::task<void> {
    try {
        char buffer[8];
        while (auto size = co_await net::async_receive(
            client, net::buffer(buffer)))
        {
            std::string_view message(+buffer, size);
            std::cout << "received<" << size << ">(" << message << ")\n";
            auto ssize = co_await net::async_send(
                client, net::const_buffer(buffer, size));
            std::cout << "sent<" << ssize << "/" << message.size()
                << ">(" << std::string_view(buffer, ssize) << ")\n";
        }
        std::cout << "client done\n";
    } catch (const std::exception& e) {
        std::cerr << "ERROR: " << e.what() << '\n';
    }
}
```

`while (auto size = co_await ...)` looks clean until you realize the error path is invisible - it is hiding inside the `try/catch` ten lines away.

Corosio^[3] `do_session` :

```

copy::task<> do_session()
{
    for (;;)
    {
        buf_.resize(4096);
        auto [ec, n] = co_await sock_.read_some(
            copy::mutable_buffer(buf_.data(), buf_.size()));
        if (ec)
            break;

        buf_.resize(n);
        auto [wec, wn] = co_await copy::write(
            sock_, copy::const_buffer(buf_.data(), buf_.size()));
        if (wec)
            break;
    }
    sock_.close();
}

```

8. Support Machinery

Claim: “std::execution provides the building blocks; users compose them into complete solutions.”

- beman.net: `demo_task.hpp` ^[2] (254) + `demo_algorithm.hpp` ^[2] (387) + `demo_scope.hpp` ^[2] (91) = 732 lines
- Corosio: `#include <boost/copy/task.hpp>` = 0 lines (library provides the task type)

732 lines of “demo” headers to run an 88-line echo server. These are not scaffolding. They implement a task type, a sender-to-coroutine bridge, structured concurrency, and an error channel adaptor - functionality that `std::execution` does not provide.

What each header implements

- `demo_task.hpp` (254 lines): task type, promise, `sender_awaiter`, operation state. This is what `std::execution::task` is supposed to be.
- `demo_algorithm.hpp` (387 lines): `when_any`, `into_error`, `into_expected`. Three sender algorithms that do not exist in `std::execution`.
- `demo_scope.hpp` (91 lines): structured concurrency scope. `counting_scope` is not in C++26.

9. Structured Concurrency

Claim: "Senders provide structured concurrency that coroutine approaches lack."

```
// beman.net - demo_scope.hpp
~scope() {
    if (0u < this->count)
        std::cerr << "ERROR: scope destroyed with live jobs: "
                    << this->count << "\n";
}
```

```
// Corosio - tcp_server.cpp
void tcp_server::start() {
    if (active_accepts_ != 0)
        detail::throw_logic_error(
            "tcp_server::start: previous session not joined");
}
```

The sender-based scope prints to stderr when destroyed with live jobs but does not block, does not join, and does not prevent the program from continuing. The coroutine-based server throws if you violate the lifecycle. Structured concurrency means enforcement, not diagnostics.

Four properties demonstrated

(a) RAII ownership. The `launcher`^[3] class is move-only and must be invoked exactly once. If destroyed without invoking, the worker returns to the pool automatically. If the coroutine setup throws, a stack guard returns the worker. Double-invocation throws `logic_error`:

```

// Corosio - tcp_server.hpp: launcher
~launcher()
{
    if (w_)
        srv_ -> push_sync(*w_);
}

template<class Executor>
void operator()(Executor const& ex, copy::task<void> task)
{
    if (!w_)
        detail::throw_logic_error();

    auto* w = std::exchange(w_, nullptr);
    srv_ -> active_push(w);

    struct guard_t
    {
        tcp_server* srv;
        worker_base* w;
        ~guard_t()
        {
            if (w)
                srv -> push_sync(*w);
        }
    } guard{srv_, w};

    // ... launch coroutine ...
    guard.w = nullptr; // dismiss on success
}

```

Compare with beman.net's spawn:

```

// beman.net - demo_scope.hpp
template <ex::sender Sender>
auto spawn(Sender&& sender) {
    ++this->count;
    new job<Sender>(this, std::forward<Sender>(sender));
}

```

Raw `new`. No corresponding `delete` in the spawn path. The receiver's `complete()` calls `delete this->state` later.

(b) Hierarchical cancellation. Three levels of stop token propagation:

```
server.stop()
-> impl_>stop.request_stop()           // stops accept loops
-> do_accept checks stop_requested() // exits accept loop
-> do_stop() iterates active workers
-> w->stop_.request_stop()             // stops each worker
```

beman.net's `demo::scope` has `stop()` but no hierarchy. Workers do not receive individual stop tokens.

(c) Join semantics. `tcp_server::join()` blocks until all accept loops complete:

```
// Corosio - tcp_server.cpp
void tcp_server::join()
{
    std::unique_lock lock(impl_>join_mutex);
    impl_>join_cv.wait(lock,
        [this] { return active_accepts_ == 0; });
}
```

`demo::scope` has no `join()`. Its destructor prints an error but does not block.

(d) Bounded resources. Workers are preallocated via `set_workers()`. No `new` per connection. The pool is fixed. `demo::scope::spawn` calls `new job<Sender>(...)` per spawn - unbounded heap allocation.

10. TCP Server: Full Program

Claim: "A sender-based networking library is as straightforward to use as a callback-based one."

beman.net^[2] `main()` :

```

int main() {
    std::cout << std::unitbuf;
    std::cout << "example server\n";

    try {
        demo::scope      scope;
        net::io_context context;

        scope.spawn(std::invoke(
            [](auto& scp, auto& ctxt) -> demo::task<> {
                net::ip::tcp::endpoint endpoint(
                    net::ip::address_v4::any(), 12345);
                net::ip::tcp::acceptor acceptor(ctxt, endpoint);

                while (true) {
                    try {
                        auto [stream, ep] =
                            co_await demo::when_any(
                                net::async_accept(acceptor),
                                demo::into_error(
                                    net::resume_after(
                                        ctxt.get_scheduler(), 1s),
                                    [](auto&&...) {
                                        return ::std::error_code();
                                    }));
                        std::cout << "ep=" << ep << "\n";
                        scp.spawn(make_client(std::move(stream)));
                    } catch (...) {
                        std::cout << "timer fired\n";
                    }
                }
            },
            scope,
            context));

        context.run();
    } catch (const std::exception& ex) {
        std::cout << "EXCEPTION: " << ex.what() << "\n";
        abort();
    }
}

```

Corosio^[3] `main()` :

```

int main(int argc, char* argv[])
{
    if (argc != 3)
    {
        std::cerr <<
            "Usage: echo_server <port> <max-workers>\n"
            "Example:\n"
            "    echo_server 8080 10\n";
        return EXIT_FAILURE;
    }

    int port_int = std::atoi(argv[1]);
    auto port = static_cast<std::uint16_t>(port_int);
    int max_workers = std::atoi(argv[2]);

    corosio::io_context ioc;
    echo_server server(ioc, max_workers);

    auto ec = server.bind(corosio::endpoint(port));
    if (ec)
    {
        std::cerr << "Bind failed: " << ec.message() << "\n";
        return EXIT_FAILURE;
    }

    std::cout << "Echo server listening on port " << port
        << " with " << max_workers << " workers\n";

    server.start();
    ioc.run();
    return EXIT_SUCCESS;
}

```

beman.net's `main()` requires the programmer to understand `std::invoke`, IILE parameter passing, `demo::scope`, `demo::task`, `demo::when_any`, `demo::into_error`, and exception-based error handling before writing a single line of networking code. Corosio's `main()` is a conventional object lifecycle: create, bind, start, run.

11. Postcondition Analysis

Claim: "The three-channel model correctly partitions I/O outcomes into success, error, and cancellation."

```
beman.net:  error_code -> set_error -> exception_ptr -> rethrow -> catch
Corosio:   error_code -> io_result -> structured binding -> if (ec)
```

An I/O read always produces a byte count and a status code. Both values are always present. The operation always satisfies its postcondition. beman.net routes the status code through `set_error` - the channel for postcondition violations. But the postcondition was not violated. The error code describes the *kind* of success.

This is not a naming quibble. The channel mismatch is why `into_expected` exists (to move the status code back to the value channel), why every I/O error becomes an exception (the bridge can only surface `set_error` as `rethrow`), and why `co_yield with_error` is needed (to propagate an error code that was already an error code before the three-channel model touched it). Every absurdity in the preceding sections traces to this single architectural decision.

12. Summary

	beman.net	Corosio
Error return	<code>catch (const std::exception&)</code>	<code>if (ec)</code>
I/O error mechanism	<code>make_exception_ptr</code> + <code>rethrow</code>	<code>error_code</code> value
Error propagation	<code>into_expected</code> + <code>then</code> + <code>co_yield</code>	<code>if (ec) co_return</code>
Channel mapping	error crosses boundary 3 times	direct
Cancellation	<code>catch (...)</code> for timer expiry	<code>stop_token</code>
Accept loop exit	<code>while (true)</code> (no exit)	<code>while (!stop_requested())</code>
Symmetric transfer	<code>void</code> (unbounded stack growth)	<code>coroutine_handle<></code> return
Concurrency spawn	<code>new job<Sender>(...)</code> (heap per spawn)	preallocated worker pool
Scope enforcement	<code>~scope()</code> prints to stderr	<code>start()</code> throws on violation
Scope join	none	<code>join()</code> blocks on CV
Hierarchical stop	none	3-level stop token propagation
Support machinery	732 lines of demo headers	0 (library provides task)
Full program complexity	<code>std::invoke</code> + IILE + 5 abstractions	create, bind, start, run

References

1. [P2762R2](#) - "Sender/Receiver-Based Networking" (Dietmar Kühl, 2024). <https://wg21.link/p2762r2>
2. [bemanproject/net](#) - Sender/receiver networking library. Commit ee861c7.
<https://github.com/bemanproject/net/tree/ee861c73865e71a06816dadfa3dee29ed298d63c>
3. [cppalliance/corosio](#) - Coroutine-native networking library. Commit 6d10efa.
<https://github.com/cppalliance/corosio/tree/6d10efacf6a8f96bcff314943ba30f84aa701409>
4. [cppalliance/capy](#) - Coroutine I/O primitives library. Commit 18c30d2.
<https://github.com/cppalliance/capy/tree/18c30d2197ac8804f9426005576d4f9b80a76135>